



Q1 Harshad number

10 marks

A number is called a Harshad number (or Niven number) if it is divisible by the sum of its digits. Example: $18 \rightarrow \text{digit sum} = 9, 18 \% 9 == 0$. Write a Python program to read a number N from the user and check whether it is a Harshad number. Also print all Harshad numbers between 1 and 500.

Your Answer

```
def is_harshad(n):  
    s = sum(int(d) for d in str(n))  
    return s != 0 and n % s == 0  
  
N = int(input())  
print("Harshad" if is_harshad(N) else "Not Harshad")  
  
for num in range(1, 501):  
    if is_harshad(num):  
        print(num, end=" ")
```

 Save Answer



Q2 GCD

10 marks

The following Python program is intended to find the GCD (Greatest Common Divisor) of two numbers using the Euclidean algorithm. It contains FOUR errors. Identify each error and write the corrected line.

```
a = int(input("Enter first: ")) b = int(input("Enter second: "))
while b = 0: temp = a a = b b = temp % a
print("GCD is", a)
```

🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 a = int(input("Enter first: "))
2 b = int(input("Enter second: "))
3
4 while b != 0:
5     temp = a
6     a = b
7     b = temp % a
8
9 print("GCD is", a)
```

Test Input

24
48

Output

```
Enter first: Enter second: GCD is
24
```



ing2

← Back

Q3 sum to a target value

10 marks

Complete the missing lines in the Python program below. The program finds all pairs of elements in a list that sum to a target value. Fill in all seven blanks.

```
def find_pairs(lst, target):  
    seen = ____ pairs = []  
    for num in ____:  
        complement = target - ____  
        if complement in ____:  
            pairs.append((complement, num))  
            _____.add(num)  
    return ____  
data = [int(x) for x in input("Enter numbers: ").split()]  
t = int(input("Target: "))  
result = ____ (data, t)  
print("Pairs:", result)
```



```
result = ____ (data, t)
print("Pairs:", result)
```

Your Answer

```
def find_pairs(lst, target):
    seen = set()
    pairs = []
    for num in lst:
        complement = target - num
        if complement in seen:
            pairs.append((complement, num))
        seen.add(num)
    return pairs
data = [int(x) for x in input("Enter numbers: ").split()]
t = int(input("Target: "))
result = find_pairs(data, t)
print("Pairs:", result)
```

 Save Answer



← Back

Q4 Bitwise Operators

10 marks

Trace the following Python program step by step. Write the exact output. Also state the data type of each variable after assignment.

```
a = 0b1101
b = 0o25
c = 0x1A
result = (a & b) | c
print(f"a={a}, b={b}, c={c}, result={result}")
print(bin(result), oct(result), hex(result))
print(result >> 2, result << 1, ~result)
```

Your Answer

```
a = 0b1101
b = 0o25
c = 0x1A
result = (a & b) | c
print(f"a={a}, b={b}, c={c}, result={result}")
print(bin(result), oct(result), hex(result))
print(result >> 2, result << 1, ~result)
```




Q5 binary search

10 marks

The following Python program tries to perform binary search on a rotated sorted array. It has FOUR errors. Identify each error with the line number and rewrite the correct program.

```
def rotated_binary_search(arr, target):  
    low = 0  
    high = len(arr) - 1  
    while low <= high:  
        mid = (low + high) // 2  
        if arr[mid] == target:  
            return mid  
        if arr[low] <= arr[mid]:  
            if target >= arr[low] and target < arr[mid]:  
                high = mid - 1  
            else:  
                low = mid + 1  
        else:  
            if target > arr[mid] and target <= arr[high]:  
                low = mid + 1  
            else: high = mid - 1  
    return -1  
data = [4,5,6,7,0,1,2]  
t = input("Target: ")  
result = rotated_binary_search(data, t)  
print("Found at:", result)
```



🔧 Debugging Task: Find and fix the bugs in the code shown above. Write your corrected code in the editor below.

Your Fixed Code

```
1 def rotated_binary_search(arr, target):
2     low = 0
3     high = len(arr) - 1
4     while low <= high:
5         mid = (low + high) // 2
6         if arr[mid] == target:
7             return mid
8         if arr[low] <= arr[mid]:
9             if arr[low] <= target < arr[mid]:
10                high = mid - 1
11            else:
12                low = mid + 1
13        else:
14            if arr[mid] < target <= arr[high]:
15                low = mid + 1
16            else:
17                high = mid - 1
18    return -1
19 data = [4, 5, 6, 7, 0, 1, 2]
20 t = int(input("Target: "))
21 result = rotated_binary_search(data, t)
22 print("Found at:", result)
```

✓ Submitted

Test Input

1

Output

Target: Found at: 5

🕒 Pending manual review